BÁO CÁO TỔNG HỢP OSSVS

An toàn chuỗi cung ứng phần mềm, SBOM,
phân loại CVE và khắc phục lỗ hổng

Phạm vi: OSSVS-1 đến OSSVS-4
Nền tảng: webOS OSE / Yocto
Trọng tâm: OSS, SBOM, CVE và remediation

Bao Nguyen

Tháng 8 năm 2026

Mục lục

1 Nghiên cứu an toàn chuỗi cung ứng và OSS Scanner **1**

1.1 Mục tiêu . . . 1

1.2 Software Supply Chain và bề mặt tấn công . . . 1

1.2.1 Các nhóm rủi ro chính trong chuỗi cung ứng phần mềm . . . 2

1.3 Tầm quan trọng của Dependency Management và OSS Security . . . 5

1.4 Case Study - SolarWinds / SUNBURST . . . 6

1.4.1 Diễn biến và ý nghĩa đối với chuỗi cung ứng phần mềm . . . 7

1.5 Supply Chain Management . . . 8

1.5.1 Kiểm soát thành phần phần mềm và dependency . . . 9

1.5.2 Software Composition Analysis và OSS Scanner . . . 9

1.5.3 Software Inventory và SBOM . . . 9

1.5.4 Vulnerability Management và Security Advisory . . . 10

1.5.5 Secure Build và CI/CD Security . . . 10

1.5.6 Artifact Integrity và Provenance . . . 10

1.5.7 Identity, Access Control và Secrets . . . 11

1.5.8 Vendor và Third-party Risk Management . . . 11

1.5.9 Detection, Incident Response và Remediation . . . 11

1.5.10 Mô hình quản trị tổng thể . . . 12

1.6 SCA, CVE, CPE/PURL và giới hạn nhận diện lỗ hổng . . . 12

1.6.1 CVE và vulnerability identification . . . 13

1.6.2 CPE và vai trò trong vulnerability matching . . . 13

1.6.3 PURL và định danh package theo hệ sinh thái . . . 14

1.6.4 False Positive và False Negative . . . 14

1.6.5 CVSS và sự khác biệt giữa Severity và Risk . . . 15

1.6.6 Giới hạn của SCA và nguyên tắc triage . . . 15

1.7 OWASP Dependency-Check và Snyk Open Source . . . 16

1.7.1 OWASP Dependency-Check . . . 16

1.7.2 Snyk Open Source . . . 16
1.7.3 So sánh hai công cụ . . . 17
1.8 Thực nghiệm Dependency-Check trên dự án Java . . . 17
1.8.1 Môi trường thực nghiệm . . . 18
1.8.2 Đánh giá kết quả . . . 20

2 SBOM và quản lý thành phần OSS trên WebOS/Yocto 21
2.1 Tổng quan SBOM . . . 21
2.1.1 SBOM là gì? . . . 21
2.1.2 SBOM trong WebOS/Yocto . . . 21
2.1.3 SBOM và Software Supply Chain . . . 22
2.2 SBOM Management . . . 22
2.2.1 SPDX và CycloneDX . . . 22
2.2.2 SBOM Manager . . . 22
2.2.3 meta-wr-sbom . . . 23
2.3 FOSSLight . . . 23
2.3.1 FOSSLight ecosystem . . . 23
2.3.2 FOSSLight Yocto Scanner . . . 24
2.3.3 Cơ chế hoạt động . . . 24
2.3.4 Dữ liệu đầu vào và đầu ra . . . 24
2.3.5 Giới hạn . . . 25
2.4 So sánh công cụ . . . 25
2.5 Thực nghiệm FOSSLight Yocto trên WebOS . . . 26
2.5.1 Môi trường . . . 26
2.5.2 Chuẩn bị build metadata . . . 27
2.5.3 Kiểm tra dữ liệu đầu vào . . . 27
2.5.4 Chạy scanner . . . 27
2.5.5 Kết quả . . . 28
2.5.6 Phân tích OSS Version . . . 29
2.5.7 Đánh giá . . . 30

3 Thu thập CVE và phân loại lỗ hổng 31

3.1 OSS được lựa chọn . . . 31
3.2 Thu thập CVE . . . 31
3.2.1 OpenSSL 3.0.7 . . . 31
3.2.2 libxml2 2.9.14 . . . 32
3.2.3 curl 7.82.0 . . . 32
3.3 Phân loại CVE . . . 32
3.3.1 OpenSSL – CVE-2023-0286 . . . 32
3.3.2 OpenSSL – CVE-2023-0464 . . . 33
3.3.3 libxml2 – CVE-2022-40304 . . . 33
3.3.4 curl – CVE-2023-38545 . . . 34
3.4 Quản lý lỗ hổng . . . 34
3.5 Kết luận . . . 35

4 Khắc phục lỗ hổng và xác minh cuối cùng 36
4.1 OpenSSL . . . 36
4.1.1 Phiên bản hiện tại . . . 36
4.1.2 Nâng cấp hoặc vá lỗi . . . 37
4.1.3 Kết quả build . . . 37
4.2 libxml2 . . . 37
4.2.1 Phiên bản hiện tại . . . 37
4.2.2 Nâng cấp hoặc vá lỗi . . . 38
4.2.3 Kết quả build . . . 38
4.3 curl . . . 38
4.3.1 Phiên bản hiện tại . . . 38
4.3.2 Nâng cấp hoặc vá lỗi . . . 38
4.3.3 Kết quả build . . . 39
4.4 Build lại WebOS . . . 39
4.4.1 Lệnh build . . . 39
4.4.2 Kết quả build . . . 39
4.5 Xác minh . . . 39
4.5.1 Phiên bản OSS sau khắc phục . . . 40

CHƯƠNG 1

Nghiên cứu an toàn chuỗi cung ứng và OSS Scanner

1.1 Mục tiêu

Nội dung này nhằm nghiên cứu tổng quan về Software Supply Chain Security, tập trung vào các thành phần, quy trình và công cụ tham gia trong vòng đời phát triển và phân phối phần mềm. Trong mô hình phát triển phần mềm hiện đại, một ứng dụng không còn được xây dựng hoàn toàn từ mã nguồn do một tổ chức tự phát triển mà thường được hình thành từ nhiều thành phần khác nhau, bao gồm mã nguồn nội bộ, thư viện mã nguồn mở (Open Source Software - OSS), dependency của bên thứ ba, package manager, repository, hệ thống build, CI/CD pipeline, container image, artifact repository và cơ chế phân phối hoặc cập nhật phần mềm.

Việc sử dụng rộng rãi các thành phần mã nguồn mở giúp rút ngắn thời gian phát triển, giảm chi phí và tận dụng được các thư viện đã được cộng đồng kiểm thử. Tuy nhiên, điều này đồng thời làm mở rộng bề mặt tấn công (attack surface) của hệ thống. Một lỗ hổng hoặc hành vi không an toàn trong một dependency có thể ảnh hưởng trực tiếp hoặc gián tiếp đến toàn bộ ứng dụng sử dụng dependency đó. Ngoài ra, rủi ro không chỉ xuất hiện ở bản thân mã nguồn mà còn có thể phát sinh trong quá trình lấy dependency, quản lý source code, build, kiểm thử, lưu trữ artifact và phân phối phần mềm.

1.2 Software Supply Chain và bề mặt tấn công

Software Supply Chain (chuỗi cung ứng phần mềm) là tập hợp các thành phần, con người, quy trình và hệ thống tham gia vào quá trình tạo ra, xây dựng, kiểm thử, đóng gói, phân phối và cập nhật một sản phẩm phần mềm. Chuỗi này có thể bắt đầu từ mã nguồn do đội ngũ phát triển viết, sau đó sử dụng các thư viện OSS hoặc dependency của bên thứ ba, lấy package từ các package repository, thực hiện build thông qua hệ thống CI/CD, tạo ra artifact hoặc container image và cuối cùng phân phối sản phẩm tới người dùng hoặc môi trường production.

Một software supply chain điển hình có thể bao gồm các thành phần:

- **Developer và source code:** mã nguồn được phát triển bởi đội ngũ nội bộ hoặc được lấy từ các dự án mã nguồn mở.
- **Source code repository:** Git repository hoặc các nền tảng lưu trữ source code, nơi quản lý mã nguồn, branch, commit và quyền truy cập.

- **Third-party dependencies:** thư viện, framework, package hoặc module được sử dụng để xây dựng ứng dụng.
- **Package repository:** nơi lưu trữ và phân phối package, ví dụ Maven Central đối với hệ sinh thái Java/Maven.
- **Build system:** các công cụ chịu trách nhiệm biên dịch source code, resolve dependency, đóng gói và tạo artifact.
- **CI/CD pipeline:** hệ thống tự động thực hiện build, test, security scanning và deployment.
- **Build runner/agent:** máy hoặc môi trường thực thi các job trong pipeline.
- **Artifact repository:** nơi lưu trữ các file build output như JAR, WAR, container image hoặc package.
- **Distribution và update mechanism:** cơ chế đưa phần mềm đến người dùng hoặc cập nhật phiên bản mới.
- **Secrets và credentials:** token, API key, SSH key, signing key hoặc các thông tin xác thực được sử dụng trong quá trình build và deployment.

Điểm quan trọng là mỗi thành phần trong chuỗi đều có thể trở thành một điểm xâm nhập hoặc bị thao túng. Nếu attacker kiểm soát được một mắt xích đủ quan trọng, họ có thể tác động đến artifact cuối cùng mà người dùng hoặc hệ thống downstream tin tưởng.

Ví dụ, một ứng dụng Java có thể sử dụng hàng chục hoặc hàng trăm dependency gián tiếp. Developer chỉ khai báo một dependency trực tiếp trong pom.xml, nhưng dependency đó lại phụ thuộc vào nhiều package khác. Vì vậy, việc kiểm soát software supply chain không thể chỉ dựa trên việc kiểm tra source code nội bộ mà cần xác định và quản lý toàn bộ dependency tree.

Ngoài số lượng dependency, một yếu tố quan trọng khác là transitive dependency. Đây là các dependency không được developer khai báo trực tiếp nhưng được kéo vào thông qua dependency khác. Điều này tạo ra rủi ro đáng kể vì developer có thể không nhận biết đầy đủ tất cả thành phần thực tế được đưa vào artifact cuối cùng.

1.2.1 Các nhóm rủi ro chính trong chuỗi cung ứng phần mềm

1.2.1.1 Dependency compromise

Dependency compromise xảy ra khi một thư viện hoặc package được sử dụng trong phần mềm bị xâm nhập hoặc bị phát hành phiên bản chứa mã độc. Attacker có thể tìm cách chiếm quyền tài khoản maintainer, thay đổi source code hoặc quy trình release để đưa malicious code vào package.

Nếu package bị compromise được sử dụng rộng rãi, tác động có thể lan truyền tới rất nhiều ứng dụng downstream. Đặc biệt, nguy cơ trở nên nghiêm trọng khi package được tích hợp tự động vào quá trình build mà không có bước kiểm tra nguồn gốc, integrity hoặc security scanning phù hợp.

1.2.1.2 Repository hoặc maintainer account compromise

Repository và tài khoản của maintainer là các mục tiêu có giá trị cao bởi chúng thường có quyền thay đổi source code hoặc phát hành phiên bản mới của phần mềm.

Attacker có thể khai thác:

- Mật khẩu yếu hoặc bị tái sử dụng.
- Thiếu hoặc cấu hình không đầy đủ MFA.
- Personal Access Token bị lộ.
- SSH key hoặc signing key bị compromise.
- CI/CD credentials có quyền quá mức.
- Repository permission được cấu hình không phù hợp.

Nếu attacker chiếm được quyền maintainer, họ có thể thay đổi source code, tạo release độc hại hoặc sửa cấu hình build mà vẫn khiến artifact được phát hành dưới danh nghĩa của dự án hợp pháp.

1.2.1.3 Build và CI/CD compromise

CI/CD pipeline là một thành phần đặc biệt quan trọng vì đây là nơi source code và dependency được biến đổi thành artifact có khả năng được triển khai vào production.

Các rủi ro có thể xuất hiện tại:

- Pipeline configuration.
- Build script.
- CI/CD runner.
- Environment variables.
- Secrets.
- Credentials.
- Build cache.
- Artifact generation.
- Deployment credentials.

Nếu attacker kiểm soát được build environment, họ có thể không cần sửa trực tiếp source code mà vẫn có khả năng tác động đến artifact được tạo ra. Ví dụ, malicious code có thể được chèn trong quá trình build hoặc một dependency độc hại có thể được resolve trong môi trường CI.

Do đó, bảo mật supply chain cần xem xét cả source integrity và build integrity, thay vì chỉ kiểm tra mã nguồn trước khi build.

1.2.1.4 Artifact compromise

Artifact là sản phẩm đầu ra của quá trình build, chẳng hạn như JAR, WAR, executable, package hoặc container image. Nếu artifact bị thay đổi sau khi build nhưng trước khi được phân phối, người dùng có thể nhận được một phiên bản khác với phiên bản thực sự được tạo bởi hệ thống build đáng tin cậy.

Các biện pháp như checksum, cryptographic signature, artifact signing và kiểm soát quyền truy cập repository có thể được sử dụng để tăng khả năng xác minh tính toàn vẹn của artifact.

1.2.1.5 Distribution và update compromise

Ngay cả khi source code và quá trình build được bảo vệ, cơ chế phân phối và cập nhật vẫn có thể trở thành mục tiêu tấn công.

Ví dụ, attacker có thể tìm cách:

- Thay thế package hoặc artifact trên máy chủ phân phối.
- Can thiệp vào cơ chế update.
- Phát hành phiên bản giả mạo.
- Chiếm quyền hệ thống phân phối.
- Làm cho client tải xuống artifact không đáng tin cậy.

Vì vậy, cơ chế cập nhật cần xác minh nguồn gốc và tính toàn vẹn của phiên bản mới thay vì chỉ dựa vào việc phiên bản đó có đúng tên hoặc đúng URL hay không.

1.2.1.6 Dependency Confusion

Dependency confusion là một dạng tấn công trong đó attacker lợi dụng cách package manager tìm kiếm và resolve dependency. Nếu tổ chức sử dụng package nội bộ nhưng không kiểm soát namespace hoặc source repository phù hợp, attacker có thể đăng một package cùng tên hoặc tên tương tự lên public package repository.

Trong một số trường hợp, hệ thống build có thể lựa chọn package từ public repository thay vì package nội bộ. Nếu package giả mạo chứa malicious code, code đó có thể được đưa vào quá trình build và cuối cùng xuất hiện trong sản phẩm.

Rủi ro này cho thấy việc quản lý dependency không chỉ liên quan đến phiên bản mà còn liên quan đến nguồn gốc (provenance) của package.

1.2.1.7 Typosquatting

Typosquatting khai thác lỗi đánh máy hoặc sự nhầm lẫn về tên package. Attacker tạo một package có tên gần giống với package hợp pháp, ví dụ thay đổi một ký tự hoặc sử dụng cách viết dễ nhầm.

Nếu developer vô tình lựa chọn package độc hại, malicious code có thể được đưa vào ứng dụng ngay từ bước cài đặt dependency.

1.2.1.8 Malicious hoặc compromised build dependency

Không phải tất cả rủi ro đều đến từ dependency trực tiếp của ứng dụng. Một build plugin hoặc công cụ được sử dụng trong quá trình compile, test hoặc package cũng có thể trở thành điểm tấn công.

Do đó, khi đánh giá software supply chain cần xem xét cả runtime dependency và build-time dependency/tooling.

1.3 Tầm quan trọng của Dependency Management và OSS Security

Trong các dự án hiện đại, số lượng dependency có thể tăng nhanh theo thời gian. Một dependency có thể phụ thuộc vào nhiều dependency khác, tạo thành một dependency tree phức tạp. Điều này khiến việc kiểm tra thủ công trở nên khó khăn và dễ bỏ sót.

Một dependency có thể tồn tại các vấn đề như:

- Có CVE đã được công bố.
- Đang sử dụng phiên bản cũ không còn được hỗ trợ.
- Có dependency transitively vulnerable.
- Có license không phù hợp với chính sách của tổ chức.
- Được lấy từ nguồn không đáng tin cậy.
- Không có cơ chế xác minh integrity đầy đủ.

Vì vậy, tổ chức cần có quy trình Software Composition Analysis (SCA) để tự động nhận diện thành phần OSS và đánh giá rủi ro. OSS Scanner/SCA tool thường phân tích manifest hoặc dependency tree của ứng dụng, xác định package và version đang được sử dụng, sau đó đối chiếu với các cơ sở dữ liệu lỗ hổng để phát hiện những dependency có khả năng bị ảnh hưởng.

Trong phạm vi thực hành của task này, OWASP Dependency-Check được sử dụng để thực hiện quá trình kiểm tra dependency của dự án Java/Maven. Công cụ giúp xác định các thành phần dependency và đối chiếu chúng với thông tin vulnerability đã biết, từ đó hỗ trợ developer/security engineer nhận diện các dependency cần được xem xét, cập nhật hoặc giảm thiểu rủi ro.

Kết quả scan cần được phân tích cùng với context thực tế của ứng dụng. Việc một dependency được công cụ đánh dấu không đồng nghĩa với việc ứng dụng chắc chắn có thể bị khai thác trong mọi trường hợp. Security engineer cần xem xét thêm phiên bản, thành

phần bị ảnh hưởng, phạm vi ảnh hưởng, cách dependency được sử dụng trong ứng dụng và khả năng khai thác thực tế.

Nhìn tổng thể, Software Supply Chain Security cần được triển khai theo hướng phòng thủ nhiều lớp, bao gồm quản lý dependency, kiểm soát source repository, bảo vệ tài khoản và secrets, bảo mật CI/CD, kiểm tra artifact, xác minh integrity/provenance và thực hiện OSS/SCA scanning định kỳ. Cách tiếp cận này giúp giảm nguy cơ một thành phần bên thứ ba hoặc một mắt xích trong quy trình phát triển trở thành điểm xâm nhập vào hệ thống phần mềm cuối cùng.

1.4 Case Study - SolarWinds / SUNBURST

Một trong những trường hợp điển hình cho rủi ro của Software Supply Chain là sự cố liên quan đến nền tảng SolarWinds Orion và malware được biết đến với tên SUNBURST. Sự kiện này có ý nghĩa đặc biệt trong nghiên cứu an toàn chuỗi cung ứng phần mềm bởi quá trình tấn công không chỉ hướng trực tiếp vào một hệ thống cuối, mà lợi dụng mối quan hệ tin cậy giữa nhà cung cấp phần mềm và các tổ chức sử dụng sản phẩm.

Theo cảnh báo được CISA công bố ngày 13/12/2020, cơ quan này xác nhận đang có hoạt động khai thác tích cực liên quan đến SolarWinds Orion Platform. CISA xác định phạm vi phiên bản từ 2019.4 HF 5 đến 2020.2.1 HF 1, được phát hành trong khoảng tháng 03/2020 đến tháng 06/2020.

Sau đó, CISA mô tả sự kiện là một trường hợp APT actor compromise SolarWinds Orion software supply chain. CISA cho biết tác nhân đã đưa mã độc vào nhiều phiên bản của nền tảng Orion và sử dụng phần mềm bị compromise cùng với các kỹ thuật khác để đạt được initial access vào mạng của một số tổ chức bị ảnh hưởng.

Cần phân biệt giữa dải phiên bản được CISA đưa vào cảnh báo và các phiên bản cụ thể được xác định chứa SUNBURST. Vì vậy, trong báo cáo không nên diễn đạt rằng toàn bộ dải từ 2019.4 HF 5 đến 2020.2.1 HF 1 đều chứa SUNBURST. Cách diễn đạt chính xác hơn là: CISA xác định dải phiên bản nói trên thuộc phạm vi cảnh báo về hoạt động khai thác SolarWinds Orion; việc xác định phiên bản cụ thể chứa mã độc cần dựa trên advisory và thông tin điều tra tương ứng.

1.4.1 Diễn biến và ý nghĩa đối với chuỗi cung ứng phần mềm

Bảng 1.1: Diễn biến chính của sự cố SolarWinds/SUNBURST.

Thời điểm	Sự kiện được nguồn công khai xác nhận	Ý nghĩa đối với quản trị chuỗi cung ứng
03–06/2020	Các phiên bản Orion thuộc dải được CISA nêu trong cảnh báo được phát hành trong giai đoạn này.	Tổ chức cần duy trì inventory về sản phẩm và phiên bản đang sử dụng, đồng thời có khả năng truy vết quá trình triển khai.
13/12/2020	CISA công bố cảnh báo về hoạt động khai thác tích cực đối với SolarWinds Orion.	Threat intelligence phải có khả năng được liên kết với asset inventory để nhanh chóng xác định hệ thống có khả năng bị ảnh hưởng.
13/12/2020	CISA ban hành Emergency Directive 21-01 liên quan đến SolarWinds Orion compromise đối với các cơ quan thuộc phạm vi chỉ thị.	Một supply-chain incident yêu cầu khả năng inventory, containment, investigation và remediation thay vì chỉ xử lý theo quy trình patch thông thường.
17/12/2020	CISA công bố Activity Alert AA20-352A liên quan đến APT compromise đối với các cơ quan chính phủ, hạ tầng trọng yếu và tổ chức khu vực tư nhân.	Phạm vi bảo vệ phải bao phủ toàn bộ chuỗi từ source, build, artifact đến deployment và hệ thống downstream.
12/2020 trở đi	CISA tiếp tục công bố các hướng dẫn và tài liệu phục vụ phát hiện, điều tra và khắc phục sự cố.	Supply-chain security cần kết hợp giữa phòng ngừa, phát hiện, điều tra và khôi phục.

Điểm có ý nghĩa nhất của SolarWinds/SUNBURST đối với nghiên cứu supply chain security nằm ở mô hình truyền tải niềm tin. Một tổ chức có thể triển khai phần mềm từ một nhà cung cấp được tin cậy mà không trực tiếp tương tác với source code hoặc quá trình build của nhà cung cấp. Do đó, tổ chức downstream thường phải dựa vào giả định rằng artifact do nhà cung cấp phát hành là hợp lệ và không bị can thiệp.

Khi chính chuỗi cung ứng của nhà cung cấp bị compromise, mối quan hệ tin cậy này trở thành một kênh để tác động tới downstream customer. CISA mô tả SolarWinds Orion là một thành phần của supply-chain compromise và cho biết APT actor đã sử dụng các phiên bản Orion bị bổ sung mã độc cùng với những kỹ thuật khác để thực hiện initial access vào các mạng doanh nghiệp bị nhắm mục tiêu.

Điều này dẫn đến một nhận định quan trọng: bảo mật phần mềm không thể chỉ được

đánh giá tại điểm triển khai cuối cùng. Nếu chỉ tập trung vào endpoint security, network security hoặc vulnerability scanning tại hệ thống khách hàng, tổ chức vẫn có thể không phát hiện được rủi ro bắt nguồn từ chính artifact được cung cấp bởi một bên thứ ba đáng tin cậy.

Sự cố cũng cho thấy tầm quan trọng của software inventory và khả năng truy xuất nguồn gốc. Khi một sản phẩm hoặc phiên bản được xác định có liên quan đến một supply-chain compromise, tổ chức cần nhanh chóng xác định sản phẩm đó đang được triển khai ở đâu, phiên bản nào đang được sử dụng và phạm vi hệ thống có liên quan. CISA trong các tài liệu remediation cũng nhấn mạnh việc đánh giá các hệ thống bị ảnh hưởng và điều tra hoạt động tiếp theo của tác nhân trên mạng.

Từ góc độ quản trị, SolarWinds/SUNBURST cho thấy một nguyên tắc quan trọng:

> ***Việc tin cậy nhà cung cấp phần mềm không loại bỏ nhu cầu kiểm soát Software Supply Chain.***

Do đó, bảo mật chuỗi cung ứng phải được xem là một quá trình xuyên suốt từ source code → dependency → build → artifact → distribution → deployment → monitoring, thay vì chỉ là hoạt động kiểm tra bảo mật tại một giai đoạn riêng lẻ.

1.5 Supply Chain Management

Software Supply Chain Management có thể được hiểu là quá trình xác lập và duy trì các cơ chế quản trị nhằm kiểm soát tính bảo mật, tính toàn vẹn, nguồn gốc và khả năng truy xuất của các thành phần tham gia vào vòng đời phần mềm.

Khác với cách tiếp cận chỉ tập trung vào việc tìm kiếm vulnerability, supply chain management đặt câu hỏi rộng hơn:

- Phần mềm được tạo thành từ những thành phần nào?
- Những thành phần đó có nguồn gốc từ đâu?
- Dependency nào được sử dụng trực tiếp và dependency nào được đưa vào gián tiếp?
- Thành phần được build và đóng gói trong môi trường nào?
- Artifact được tạo ra bởi quy trình nào?
- Artifact có thể được xác minh về tính toàn vẹn và nguồn gốc hay không?
- Khi một vulnerability hoặc compromise được phát hiện, tổ chức có thể xác định nhanh những hệ thống nào bị ảnh hưởng hay không?

Theo hướng tiếp cận của NIST, quản trị software supply chain cần kết hợp nhiều năng lực như SBOM, quản lý OSS, vulnerability management, provenance, secure software development và đánh giá rủi ro nhà cung cấp. Các năng lực này không hoạt động độc lập mà bổ trợ cho nhau trong suốt vòng đời phần mềm.

1.5.1 Kiểm soát thành phần phần mềm và dependency

Dependency là một trong những nguồn rủi ro cơ bản của software supply chain bởi ứng dụng hiện đại thường được xây dựng từ nhiều thư viện và thành phần bên thứ ba. Một dependency có thể tiếp tục phụ thuộc vào các thành phần khác, tạo thành transitive dependency và làm cho cấu trúc thực tế của phần mềm phức tạp hơn những gì thể hiện trực tiếp trong source code.

Vì vậy, hoạt động quản lý dependency cần bảo đảm rằng tổ chức có khả năng xác định chính xác component, version và nguồn cung cấp của các thành phần được sử dụng.

Một trong những kỹ thuật hỗ trợ mục tiêu này là dependency pinning, trong đó phiên bản dependency được xác định rõ thay vì để quá trình build tự động lựa chọn phiên bản không xác định. Việc kiểm soát version giúp tăng khả năng tái lập build, hỗ trợ điều tra khi xảy ra sự cố và làm rõ artifact được tạo ra từ tập hợp dependency nào.

Tuy nhiên, dependency pinning không đồng nghĩa với việc giữ nguyên dependency trong thời gian dài. Một phiên bản được cố định nhưng đã xuất hiện vulnerability vẫn có thể trở thành nguồn rủi ro. Do đó, pinning cần được kết hợp với dependency review, vulnerability monitoring và quy trình cập nhật có kiểm soát.

NIST cũng khuyến nghị các tổ chức phải quan tâm đến tính toàn vẹn và provenance của OSS components, đồng thời sử dụng các cơ chế phân tích thành phần để nhận diện vulnerability trong phần mềm.

1.5.2 Software Composition Analysis và OSS Scanner

Khi số lượng dependency tăng lên, việc xác định thủ công các thành phần và vulnerability trở nên khó duy trì. Software Composition Analysis (SCA) được sử dụng để tự động hóa một phần quá trình này.

Về bản chất, SCA thực hiện việc nhận diện các thành phần phần mềm có trong ứng dụng, xác định phiên bản và đối chiếu thông tin với các nguồn dữ liệu vulnerability. Kết quả có thể được sử dụng để xác định dependency cần cập nhật hoặc đánh giá thêm.

Trong phạm vi thực hành của task này, OWASP Dependency-Check được sử dụng để phân tích dependency của dự án Java/Maven. Công cụ giúp kiểm tra các dependency được xác định từ project và đối chiếu chúng với thông tin vulnerability đã biết. Kết quả thực nghiệm được trình bày tại Mục 2.10.

Tuy nhiên, kết quả của SCA cần được xem là đầu vào cho quá trình đánh giá rủi ro, thay vì được coi là kết luận rằng hệ thống chắc chắn có thể bị khai thác.

1.5.3 Software Inventory và SBOM

Một yêu cầu quan trọng của supply chain management là phải duy trì inventory về các thành phần phần mềm. Nếu tổ chức không biết phần mềm của mình được cấu thành từ những component nào, việc đánh giá tác động khi xuất hiện một vulnerability mới sẽ trở nên chậm và thiếu chính xác.

Software Bill of Materials (SBOM) cung cấp một phương thức có cấu trúc để mô tả các thành phần và mối quan hệ supply-chain trong một sản phẩm phần mềm. NIST mô tả SBOM như một bản ghi chính thức về các thành phần và quan hệ supply chain được sử dụng để xây dựng phần mềm.

Giá trị của SBOM không chỉ nằm ở việc liệt kê dependency. Khi được kết hợp với vulnerability database và asset inventory, SBOM có thể hỗ trợ quá trình xác định nhanh phạm vi ảnh hưởng của một vulnerability mới.

1.5.4 Vulnerability Management và Security Advisory

Software supply chain là một môi trường thay đổi liên tục. Một phần mềm có thể không có vulnerability đã biết tại thời điểm phát hành nhưng sau đó có thể xuất hiện vulnerability mới trong chính dependency của nó. Vì vậy, security assessment không nên chỉ được thực hiện một lần tại thời điểm build.

Quản lý vulnerability cần bao gồm khả năng tiếp nhận và phân tích các nguồn thông tin như CVE, security advisory, vendor bulletin và thông báo bảo mật từ OSS project. Thông tin này sau đó cần được liên kết với software inventory để xác định mức độ liên quan đến hệ thống đang vận hành.

NIST nhấn mạnh việc tích hợp SBOM, vulnerability database và các cơ chế báo cáo vulnerability nhằm rút ngắn thời gian tổ chức nhận biết các vulnerability mới được công bố.

1.5.5 Secure Build và CI/CD Security

Một software supply chain không kết thúc ở source repository. Source code sau đó được đưa qua hệ thống build và CI/CD để tạo thành artifact. Vì vậy, build environment và CI/CD pipeline cũng phải được xem là các thành phần cần bảo vệ.

Các thành phần cần được kiểm soát bao gồm build server, CI/CD runner, pipeline configuration, build script, service account, credential và secret được sử dụng trong quá trình build hoặc deployment.

Rủi ro ở đây có đặc điểm khác với dependency vulnerability. Một dependency vulnerable có thể tồn tại trong source tree, trong khi một pipeline compromise có thể tác động trực tiếp đến quá trình tạo artifact.

NIST SSDF nhấn mạnh việc tích hợp các thực hành phát triển phần mềm an toàn vào SDLC.

1.5.6 Artifact Integrity và Provenance

Sau khi quá trình build hoàn tất, artifact trở thành đối tượng cần được bảo vệ. Artifact có thể là JAR, WAR, executable, package, container image hoặc một dạng binary khác.

Hai thuộc tính quan trọng cần được quan tâm là integrity và provenance.

- **Integrity:** artifact có bị thay đổi sau khi được tạo hay không?
- **Provenance:** artifact được tạo từ source nào, dependency nào và quy trình build nào?

Các cơ chế như cryptographic hash, digital signature và provenance metadata có thể hỗ trợ việc xác minh những thuộc tính này.

1.5.7 Identity, Access Control và Secrets

Con người và tài khoản kỹ thuật là một phần của software supply chain. Developer account, maintainer account, CI/CD service account và signing key đều có thể cung cấp quyền truy cập tới những thành phần quan trọng của chuỗi.

Do đó, Identity and Access Management (IAM) cần được triển khai theo nguyên tắc least privilege.

1.5.8 Vendor và Third-party Risk Management

Đối với phần mềm do bên thứ ba cung cấp, tổ chức downstream không kiểm soát trực tiếp toàn bộ quá trình phát triển. Vì vậy, third-party risk management trở thành một thành phần quan trọng của supply chain security.

Việc đánh giá nhà cung cấp có thể xem xét:

- Quy trình secure software development
- Quản lý vulnerability
- Security advisory
- SBOM
- Bảo vệ artifact
- OSS management
- Provenance

NIST cũng đề xuất tăng cường vendor risk assessment và xác minh tính toàn vẹn của phần mềm từ nhà cung cấp.

1.5.9 Detection, Incident Response và Remediation

Preventive controls không thể bảo đảm tuyệt đối rằng supply chain sẽ không bị compromise. Vì vậy, tổ chức cần duy trì năng lực phát hiện và ứng phó với sự cố.

Quy trình xử lý cần trả lời:

- Phạm vi ảnh hưởng là gì?

- Compromise đã lan chưa?
- Remediation cần thực hiện ở đâu?

Trong case SolarWinds, CISA đã cung cấp hướng dẫn riêng cho việc đánh giá và khắc phục sự cố, cho thấy tính phức tạp của supply-chain incident.

1.5.10 Mô hình quản trị tổng thể

Software Supply Chain Management có thể được mô hình hóa như một chu trình:

> ***Component Management → Inventory/SBOM → Vulnerability Assessment → Secure Build → Artifact Verification → Distribution Control → Monitoring → Incident Response → Remediation → Continuous Improvement***

Mỗi lớp giải quyết một nhóm rủi ro khác nhau.

Không một biện pháp đơn lẻ nào có thể bảo vệ toàn bộ software supply chain. Do đó, cách tiếp cận phù hợp là defense-in-depth, kết hợp nhiều lớp kiểm soát xuyên suốt vòng đời phần mềm.

1.6 SCA, CVE, CPE/PURL và giới hạn nhận diện lỗ hổng

Trong hệ thống phần mềm hiện đại, việc sử dụng các thư viện mã nguồn mở và thành phần bên thứ ba làm cho quá trình đánh giá bảo mật không thể chỉ dựa trên việc kiểm tra mã nguồn do tổ chức tự phát triển. Một ứng dụng có thể bao gồm nhiều dependency trực tiếp và gián tiếp, trong đó mỗi dependency có thể tiếp tục phụ thuộc vào các thành phần khác. Vì vậy, cần có cơ chế tự động để xác định thành phần phần mềm, phiên bản đang được sử dụng và các vulnerability đã được công bố có khả năng liên quan.

Software Composition Analysis (SCA) là nhóm kỹ thuật được sử dụng cho mục đích này. Về tổng quát, SCA thực hiện ba nhiệm vụ chính: nhận diện thành phần phần mềm, xác định phiên bản/định danh của thành phần và đối chiếu thông tin đó với các nguồn dữ liệu vulnerability. Kết quả của quá trình phân tích thường bao gồm danh sách dependency, phiên bản, vulnerability liên quan, mức độ nghiêm trọng và thông tin phục vụ remediation.

Đối với một dự án Maven, quá trình này không chỉ giới hạn ở các dependency được khai báo trực tiếp trong pom.xml. Một dependency trực tiếp có thể kéo theo nhiều transitive dependencies, do đó một SCA tool cần có khả năng xây dựng hoặc phân tích dependency graph để xác định các thành phần thực sự xuất hiện trong software composition.

1.6.1 CVE và vulnerability identification

Common Vulnerabilities and Exposures (CVE) là hệ thống định danh các vulnerability đã được công bố. Mỗi vulnerability được gán một CVE ID nhằm cung cấp một tham chiếu thống nhất giữa các nhà cung cấp, nhà nghiên cứu và tổ chức sử dụng thông tin bảo mật. NVD mô tả CVE là một hệ thống định danh các vulnerability gắn với những code base cụ thể như ứng dụng phần mềm hoặc thư viện.

Cần phân biệt giữa CVE và vulnerability database. CVE chủ yếu cung cấp định danh và thông tin vulnerability được công bố, trong khi NVD thực hiện quá trình enrichment đối với các CVE được công bố, bổ sung các thông tin như CVSS, CWE và CPE applicability statements.

Do đó, trong một quy trình SCA, việc phát hiện một CVE không đơn thuần là tìm kiếm chuỗi ký tự CVE-xxxx-xxxx. Công cụ cần xác định liệu dependency cụ thể đang được sử dụng có thực sự tương ứng với sản phẩm và phiên bản nằm trong phạm vi ảnh hưởng của vulnerability hay không.

Có thể khái quát quá trình như sau:

$$\textit{Dependency} \rightarrow \textit{Product/Package Identification} \rightarrow \textit{Version Identification} \rightarrow \textit{Vulnerability Matching} \rightarrow \textit{Triage}$$

Trong đó, bước vulnerability matching là một trong những nguồn gây ra sai lệch đáng chú ý nhất trong automated SCA.

1.6.2 CPE và vai trò trong vulnerability matching

Common Platform Enumeration (CPE) là một scheme định danh có cấu trúc dành cho các hệ thống, phần mềm và package. NVD sử dụng CPE 2.3 trong quá trình xác định các sản phẩm bị ảnh hưởng bởi một vulnerability thông qua các CPE applicability statements.

Đối với Dependency-Check, quá trình nhận diện dependency dựa trên các loại evidence như vendor, product và version. Công cụ thu thập evidence từ nhiều nguồn, sau đó sử dụng cơ chế matching để xác định CPE tương ứng. Khi CPE được xác định, công cụ có thể sử dụng thông tin vulnerability được liên kết với CPE đó để đưa các CVE tương ứng vào báo cáo.

Cách tiếp cận này cho phép tự động hóa vulnerability detection ở quy mô lớn, nhưng đồng thời tạo ra một giới hạn quan trọng: kết quả matching phụ thuộc vào chất lượng của thông tin nhận diện.

Ví dụ, tên package trong một manifest hoặc JAR có thể không biểu diễn đầy đủ product identity theo cách mà vulnerability database sử dụng. Một component có thể có tên tương tự với một sản phẩm khác, hoặc metadata của package không đủ để xác định chính xác vendor/product/version.

Do đó, việc một scanner tạo ra một CPE match không nên được hiểu ngay là bằng chứng rằng ứng dụng chắc chắn bị ảnh hưởng bởi vulnerability tương ứng. Cần tiếp tục kiểm tra

affected version range, vendor advisory, changelog, patch information và cách component được sử dụng trong ứng dụng.

1.6.3 PURL và định danh package theo hệ sinh thái

Bên cạnh CPE, Package URL (PURL) là một phương thức định danh package được thiết kế để biểu diễn các package thuộc nhiều package ecosystem khác nhau. PURL có cấu trúc cho phép mô tả type, namespace, name, version và các thành phần bổ sung của package.

Khác với CPE, vốn được sử dụng rộng rãi trong hệ thống nhận diện sản phẩm và vulnerability applicability của NVD, PURL tập trung mạnh vào package identity trong hệ sinh thái phần mềm. Điều này đặc biệt hữu ích trong bối cảnh SCA vì một package có thể được định danh theo ecosystem tương ứng, chẳng hạn Maven, npm, PyPI hoặc các package ecosystem khác.

PURL vì vậy có thể hỗ trợ việc chuẩn hóa thông tin component giữa các công cụ khác nhau. Tuy nhiên, cũng cần lưu ý rằng một identifier chính xác không tự động chứng minh vulnerability applicability. Identifier giúp xác định component; việc component có bị ảnh hưởng hay không vẫn phụ thuộc vào vulnerability record, version range, patch status và các điều kiện ảnh hưởng cụ thể.

1.6.4 False Positive và False Negative

Automated SCA không thể đảm bảo kết quả tuyệt đối. OWASP Dependency-Check ghi nhận rằng phương pháp phân tích dựa trên evidence có thể tạo ra cả false positives và false negatives.

False positive xảy ra khi công cụ xác định một dependency là vulnerable nhưng sau khi kiểm tra thêm cho thấy kết luận đó không áp dụng cho component hoặc cách triển khai thực tế.

- **Các nguyên nhân có thể bao gồm:**
- Dependency bị match với product/CPE không chính xác
- Phiên bản được nhận diện không đầy đủ
- Phạm vi ảnh hưởng trong database rộng hơn thực tế triển khai
- Vendor đã backport security patch nhưng version không phản ánh rõ
- Vulnerable functionality tồn tại nhưng không được sử dụng trong application

Ngược lại, false negative xảy ra khi vulnerability thực tế tồn tại nhưng scanner không phát hiện được. Điều này có thể xảy ra khi:

- Dependency không được nhận diện
- Metadata không đủ để matching
- Vulnerability chưa có trong database

- Mapping product/version chưa chính xác
- Công cụ không hỗ trợ dạng artifact cụ thể

Do đó, việc không có cảnh báo từ SCA không đồng nghĩa với việc hệ thống an toàn. Kết quả scan luôn phụ thuộc vào coverage và chất lượng dữ liệu đầu vào.

1.6.5 CVSS và sự khác biệt giữa Severity và Risk

Common Vulnerability Scoring System (CVSS) cung cấp phương pháp chuẩn hóa để đánh giá mức độ nghiêm trọng (severity) của vulnerability. Điểm CVSS nằm trong khoảng từ 0 đến 10 và được tính dựa trên các metric được chuẩn hóa.

Một điểm quan trọng:

CVSS severity không đồng nghĩa với risk thực tế của hệ thống.

CVSS chỉ phản ánh mức độ nghiêm trọng của vulnerability trong điều kiện chuẩn hóa, không phản ánh môi trường triển khai cụ thể.

Ví dụ, cùng một CVE có CVSS cao nhưng:

- Hệ thống A expose trực tiếp ra Internet → risk cao
- Hệ thống B không sử dụng chức năng bị ảnh hưởng → risk thấp

Do đó, CVSS chỉ nên được dùng như một yếu tố trong prioritization, kết hợp với context hệ thống.

1.6.6 Giới hạn của SCA và nguyên tắc triage

Kết quả SCA cần được xem là đầu vào cho quá trình security triage, không phải kết luận cuối cùng.

Một quy trình triage hợp lý:

Scanner Finding → Component Verification → Version Verification → Advisory Verification → Applicability Assessment → Risk Prioritization → Remediation

Trong đó:

- **Component Verification:** xác nhận dependency có đúng thực tế
- **Version Verification:** xác nhận version và affected range
- **Advisory Verification:** đối chiếu vendor advisory

- **Applicability Assessment:** đánh giá mức độ ảnh hưởng thực tế
- **Risk Prioritization:** xác định mức độ ưu tiên xử lý
- **Remediation:** upgrade, patch hoặc mitigation

Giá trị của SCA nằm ở việc chuyển software composition thành dữ liệu phục vụ quyết định bảo mật, không phải thay thế hoàn toàn phân tích con người.

1.7 OWASP Dependency-Check và Snyk Open Source

Trong phạm vi nghiên cứu OSS Scanner, OWASP Dependency-Check và Snyk Open Source là hai công cụ phổ biến để phân tích dependency và phát hiện vulnerability. Tuy nhiên, hai công cụ có cách tiếp cận khác nhau về kiến trúc và mục tiêu sử dụng.

Dependency-Check là công cụ mã nguồn mở thuộc OWASP, tập trung vào việc xác định component có vulnerability đã biết. Công cụ hỗ trợ nhiều môi trường như CLI, Maven, Gradle và CI/CD. Cơ chế hoạt động dựa trên việc thu thập evidence và thực hiện CPE matching để liên kết với dữ liệu vulnerability.

Snyk Open Source được thiết kế theo hướng developer-centric, tập trung vào việc phát hiện vulnerability trong dependency và hỗ trợ remediation. Công cụ phân tích cả direct và transitive dependencies và cung cấp dependency graph để hỗ trợ hiểu rõ nguồn gốc của vulnerability.

1.7.1 OWASP Dependency-Check

Dependency-Check hoạt động theo mô hình component identification → CPE matching → vulnerability correlation. Đối với Java/Maven, công cụ phân tích metadata từ pom.xml, Manifest và artifact để thu thập evidence.

Điểm quan trọng là Dependency-Check không dựa vào một nguồn thông tin duy nhất mà kết hợp nhiều evidence để tăng độ chính xác. Tuy nhiên, kết quả vẫn cần được đánh giá thủ công do phụ thuộc vào chất lượng metadata và mapping.

Công cụ phù hợp với môi trường build system vì có thể tích hợp trực tiếp vào Maven pipeline, giúp phát hiện vulnerability ngay trong quá trình build.

1.7.2 Snyk Open Source

Snyk Open Source hỗ trợ quét dependency thông qua CLI, IDE, Web UI và CI/CD integration.

Đối với Java/Maven, Snyk sử dụng Maven project structure để xây dựng dependency tree và phân tích cả direct và transitive dependencies.

Một điểm nổi bật là Snyk cung cấp dependency path, giúp xác định chính xác dependency nào đã kéo vulnerability vào project. Điều này giúp việc remediation trở nên rõ ràng hơn.

Ngoài vulnerability detection, Snyk còn hỗ trợ license compliance, mở rộng phạm vi từ security sang governance.

1.7.3 So sánh hai công cụ

Bảng 1.2: So sánh OWASP Dependency-Check và Snyk Open Source.

Tiêu chí	OWASP Dependency-Check	Snyk Open Source
Định hướng	Open-source SCA, tập trung vào vulnerability detection	Developer-centric SCA, hỗ trợ remediation
Cơ chế	Evidence → CPE → CVE mapping	Dependency graph → vulnerability database
Direct dependency	Có	Có
Transitive dependency	Có	Có, kèm dependency path
Data source	NVD + nguồn bổ sung	Snyk vulnerability database
CPE	Có vai trò quan trọng	Không sử dụng theo cách tương tự
Maven	Có plugin	Có CLI support
CI/CD	Có	Có
License analysis	Không	Có
Remediation	Hạn chế	Có đề xuất fix
Ưu điểm	Open-source, dễ tích hợp build	UX tốt, dependency graph rõ ràng
Hạn chế	Cần manual review, dễ false positive/negative	Phụ thuộc hệ sinh thái Snyk

Hai công cụ không hoàn toàn thay thế nhau mà phục vụ các mục tiêu khác nhau: Dependency-Check phù hợp cho tích hợp build pipeline, trong khi Snyk phù hợp cho workflow phát triển và quản lý rủi ro.

1.8 Thực nghiệm Dependency-Check trên dự án Java

Thực nghiệm được thực hiện trên một Java/Maven sample project có pom.xml. OWASP Dependency-Check được chạy thông qua Maven plugin để phân tích các dependency của project và đối chiếu với cơ sở dữ liệu vulnerability.

1.8.1 Môi trường thực nghiệm

Bảng 1.3: Môi trường thực nghiệm OWASP Dependency-Check.

Thành phần	Giá trị
OS	Kali Linux
Project	dependency-check-lab
Build system	Apache Maven
Dependency-Check Maven Plugin	12.2.2
Java project	Maven/JAR
Report	target/dependency-check-report.html

Lệnh thực hiện:

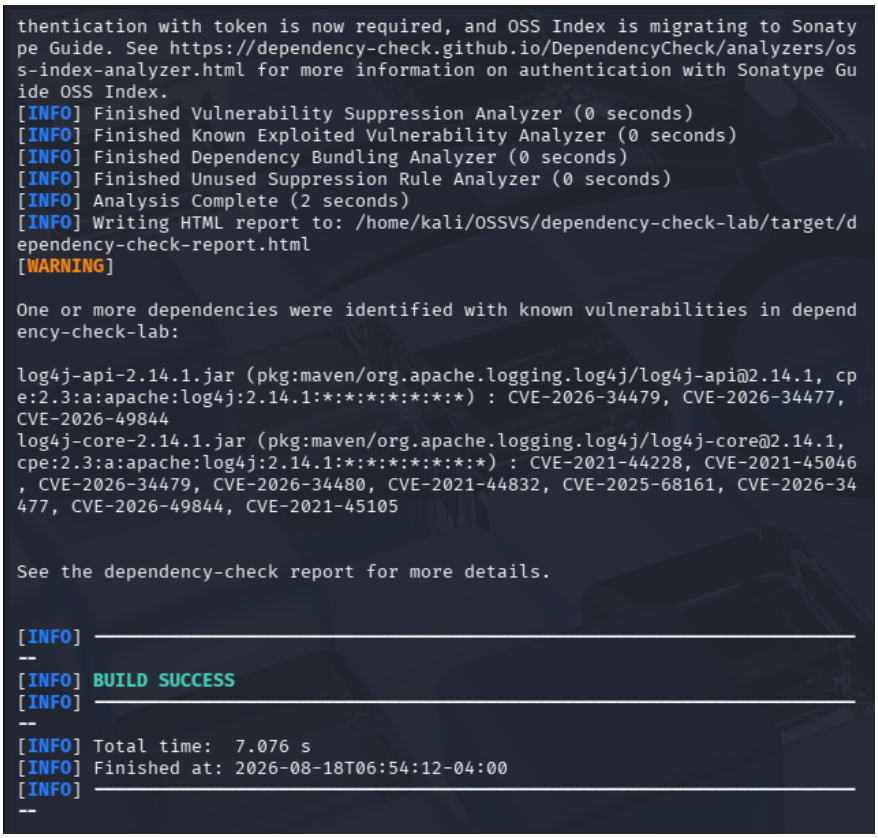
```
mvn org.owasp:dependency-check-maven:check
```

Kết quả thực nghiệm cho thấy Dependency-Check đã hoàn thành quá trình phân tích và tạo báo cáo:

```
[INFO] Analysis Started
...
[INFO] Analysis Complete (2 seconds)
[INFO] Writing HTML report to:
/home/kali/OSSVS/dependency-check-lab/target/dependency-check-report.html
```

Quá trình Maven kết thúc với:

```
[INFO] BUILD SUCCESS
```



Hình 1.1: Kết quả chạy OWASP Dependency-Check bằng Maven.

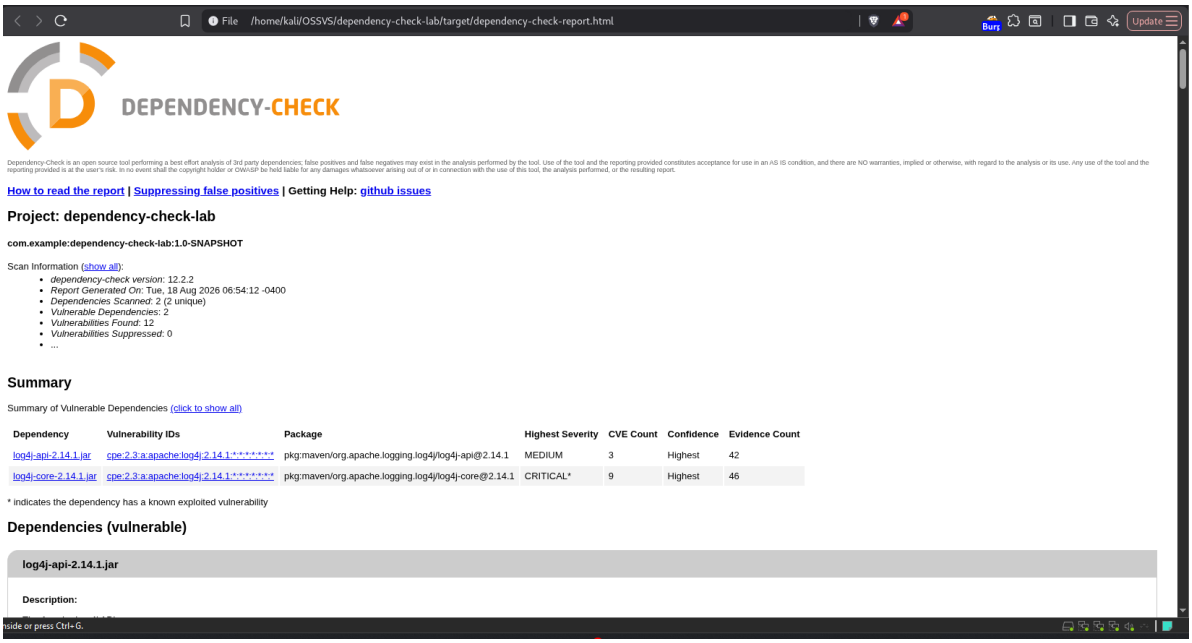
Dependency-Check phát hiện các dependency có vulnerability đã biết. Kết quả nổi bật gồm:

Bảng 1.4: Các lỗ hổng được Dependency-Check phát hiện.

Dependency	Version	CVE được phát hiện
log4j-api	2.14.1	CVE-2026-34479, CVE-2026-34477, CVE-2026-49844
log4j-core	2.14.1	CVE-2021-44228, CVE-2021-45046, CVE-2021-44832, CVE-2021-45105, CVE-2025-68161, CVE-2026-34479, CVE-2026-34480, CVE-2026-34477, CVE-2026-49844

Report HTML được tạo tại:

```
target/dependency-check-report.html
```



Hình 1.2: Báo cáo HTML do OWASP Dependency-Check tạo.

1.8.2 Đánh giá kết quả

Thực nghiệm chứng minh Dependency-Check có thể tự động nhận diện dependency của Maven project, xác định phiên bản và đối chiếu với vulnerability database để phát hiện các CVE liên quan. Kết quả scan cần tiếp tục được triage thủ công để xác định mức độ ảnh hưởng thực tế, do scanner có thể tồn tại false positive hoặc false negative.

CHƯƠNG 2

SBOM và quản lý thành phần OSS trên WebOS/Yocto

2.1 Tổng quan SBOM

2.1.1 SBOM là gì?

Software Bill of Materials (SBOM) là danh sách có cấu trúc mô tả các thành phần phần mềm được sử dụng trong một sản phẩm hoặc hệ thống. SBOM cung cấp thông tin về các thành phần như tên phần mềm, phiên bản, giấy phép, nguồn cung cấp và mối quan hệ giữa các dependency.

Đối với các sản phẩm sử dụng nhiều thành phần Open Source Software (OSS), SBOM giúp xây dựng inventory về các thành phần được tích hợp vào sản phẩm. Từ inventory này, tổ chức có thể xác định phiên bản đang sử dụng, kiểm tra yêu cầu về license, theo dõi nguồn gốc của component và hỗ trợ quá trình đánh giá vulnerability.

SBOM thường được biểu diễn dưới các định dạng tiêu chuẩn như SPDX và CycloneDX. Các định dạng này giúp SBOM có thể được trao đổi và sử dụng bởi nhiều công cụ khác nhau trong quá trình phát triển, kiểm thử và quản lý sản phẩm phần mềm.

2.1.2 SBOM trong WebOS/Yocto

WebOS sử dụng hệ thống build dựa trên Yocto/OpenEmbedded. Các thành phần phần mềm được tích hợp vào WebOS được quản lý thông qua các recipe và metadata của quá trình build. Recipe chứa các thông tin quan trọng về package như tên, phiên bản, license và nguồn mã nguồn.

Trong quá trình Yocto build, các thông tin này có thể được sử dụng để tạo dữ liệu BOM và inventory về các package được đưa vào WebOS image. Đối với FOSSLight Yocto Scanner, các dữ liệu như buildhistory, bom.json, installed-package-names.txt và installed-packages.txt được sử dụng làm đầu vào cho quá trình phân tích OSS.

FOSSLight Yocto khai thác metadata của Yocto recipe để xác định thông tin về OSS thay vì chỉ dựa trên việc nhận diện trực tiếp từ binary. Các thông tin như package name, version, license và source location được sử dụng để xây dựng FOSSLight Report.

Do đó, trong môi trường WebOS/Yocto, SBOM có vai trò cung cấp inventory về các OSS được tích hợp trong quá trình build, từ đó hỗ trợ quản lý component và theo dõi thành phần phần mềm của sản phẩm.

2.1.3 SBOM và Software Supply Chain

Software supply chain bao gồm các thành phần và quy trình liên quan đến việc phát triển, lấy dependency, build và phân phối một sản phẩm phần mềm. Đối với WebOS, các OSS có thể được đưa vào sản phẩm thông qua Yocto recipe và các dependency trong quá trình build.

SBOM cung cấp thông tin về các component xuất hiện trong sản phẩm và phiên bản tương ứng. Điều này giúp duy trì khả năng truy xuất nguồn gốc của các thành phần phần mềm trong toàn bộ quá trình phát triển và build.

Trong quản lý Software Supply Chain, SBOM có thể được sử dụng để hỗ trợ ba nhóm công việc chính là OSS inventory, license compliance và vulnerability management. Khi một vulnerability mới được công bố, inventory từ SBOM giúp xác định sản phẩm có sử dụng component bị ảnh hưởng hay không và đang sử dụng phiên bản nào.

Đối với WebOS, việc kết hợp Yocto build metadata với SBOM và các công cụ phân tích OSS như FOSSLight Yocto giúp xây dựng inventory về các thành phần Open Source được tích hợp vào sản phẩm. Đây là cơ sở để tiếp tục thực hiện các hoạt động quản lý OSS và đánh giá security trong các giai đoạn tiếp theo.

2.2 SBOM Management

2.2.1 SPDX và CycloneDX

SPDX và CycloneDX là hai định dạng phổ biến được sử dụng để biểu diễn Software Bill of Materials (SBOM). Chúng cung cấp cấu trúc thống nhất để mô tả các thành phần phần mềm, phiên bản, license và các thông tin liên quan trong một sản phẩm.

SPDX (Software Package Data Exchange) là một chuẩn được phát triển nhằm trao đổi thông tin về các thành phần phần mềm và license. SPDX được sử dụng rộng rãi trong hệ sinh thái Linux và các quy trình quản lý Open Source Software. Trong môi trường Yocto, SPDX có thể được sử dụng để biểu diễn thông tin về các package được tạo ra trong quá trình build.

CycloneDX là một chuẩn SBOM tập trung vào việc mô tả component và dependency relationship. Định dạng này thường được sử dụng trong các quy trình Software Composition Analysis (SCA) và DevSecOps.

Hai định dạng có thể được sử dụng làm dữ liệu đầu vào hoặc đầu ra cho các công cụ quản lý SBOM. Việc sử dụng định dạng chuẩn giúp SBOM có thể được trao đổi giữa các công cụ khác nhau thay vì phụ thuộc vào một công cụ cụ thể.

2.2.2 SBOM Manager

SBOM Manager là công cụ được sử dụng để quản lý và truy vấn SBOM sau khi SBOM đã được tạo. Công cụ hỗ trợ làm việc với các định dạng SBOM như SPDX và CycloneDX,

cho phép import SBOM vào repository và thực hiện truy vấn thông tin component.

Quy trình sử dụng có thể gồm việc khởi tạo repository, import SBOM và truy vấn component theo project. Ví dụ, một SBOM SPDX của WebOS có thể được import vào project để quản lý tập trung và kiểm tra sự xuất hiện của một component hoặc phiên bản cụ thể.

SBOM Manager cũng có thể được kết hợp với công cụ vulnerability scanner bên ngoài. Tuy nhiên, chức năng quản lý SBOM và vulnerability detection cần được phân biệt. SBOM cung cấp thông tin inventory về component, trong khi vulnerability scanner sử dụng thông tin này để xác định các vulnerability liên quan. Việc phát hiện CVE cũng không đồng nghĩa với việc component chắc chắn cần được cập nhật ngay mà cần đánh giá thêm về phiên bản bị ảnh hưởng, khả năng khai thác và mức độ ảnh hưởng đối với sản phẩm.

SBOM Manager được nghiên cứu với vai trò là thành phần quản lý SBOM, hỗ trợ giai đoạn sau khi SBOM được tạo từ WebOS/Yocto build.

2.2.3 meta-wr-sbom

meta-wr-sbom là một layer dành cho Yocto, được sử dụng để hỗ trợ tạo SBOM từ thông tin của quá trình build. Công cụ này tích hợp với hệ thống Yocto thay vì phân tích sản phẩm sau khi đã build xong.

Điểm quan trọng của cách tiếp cận này là SBOM được tạo dựa trên build metadata của Yocto. Do đó, thông tin về component và phiên bản có thể được liên kết với các package và recipe được sử dụng trong quá trình build.

meta-wr-sbom được nghiên cứu như một giải pháp tạo SBOM tích hợp với Yocto, trong khi FOSSLight Yocto được sử dụng cho mục tiêu phân tích OSS và xác định thông tin component từ WebOS build. Hai công cụ có vai trò khác nhau và không nên được xem là các công cụ thay thế trực tiếp cho nhau.

2.3 FOSSLight

2.3.1 FOSSLight ecosystem

FOSSLight là một hệ thống hỗ trợ quản lý và phân tích Open Source Software (OSS) trong quá trình phát triển phần mềm. Hệ sinh thái FOSSLight bao gồm các thành phần phục vụ việc thu thập thông tin OSS, xác định component, quản lý thông tin license và xây dựng OSS inventory.

FOSSLight được quan tâm chủ yếu ở khả năng phân tích các thành phần OSS được sử dụng trong WebOS. FOSSLight có thể sử dụng thông tin từ source code, dependency, binary và build metadata để xác định các component phần mềm.

Đối với môi trường Yocto, FOSSLight Yocto Scanner là thành phần chuyên biệt được sử dụng để khai thác metadata được tạo ra trong quá trình Yocto build. Kết quả phân tích được tổng hợp thành FOSSLight Report, trong đó có các thông tin như OSS Name, OSS

Version, License và Download Location.

2.3.2 FOSSLight Yocto Scanner

FOSSLight Yocto Scanner là công cụ được thiết kế để phân tích OSS trong các hệ thống sử dụng Yocto. Khác với phương pháp chỉ quét trực tiếp filesystem của sản phẩm, scanner sử dụng các metadata được tạo ra trong quá trình Yocto build để xác định các package và thông tin OSS tương ứng.

Các dữ liệu quan trọng được sử dụng bởi scanner gồm buildhistory, bom.json, installed-package-names.txt và installed-packages.txt. Các dữ liệu này cung cấp thông tin về package, version và các thành phần được đưa vào WebOS image.

FOSSLight Yocto phù hợp với WebOS vì WebOS sử dụng hệ thống build dựa trên Yocto. Việc khai thác build metadata giúp duy trì mối liên hệ giữa Yocto recipe, package được build và OSS được tích hợp vào sản phẩm.

2.3.3 Cơ chế hoạt động

Trong quá trình Yocto build, recipe cung cấp các thông tin như package name, package version, license và source location. Các thông tin này được sử dụng để tạo dữ liệu BOM và build history.

FOSSLight Yocto Scanner sử dụng các dữ liệu này để xác định và tổng hợp thông tin OSS. Trong đó, thông tin từ recipe có thể được sử dụng để xác định tên, phiên bản, license và nguồn của component. Kết quả được đưa vào FOSSLight Report để phục vụ việc kiểm tra và quản lý OSS.

Scanner cũng hỗ trợ bổ sung thông tin bằng source analysis, binary analysis hoặc file sbom-info.yaml khi metadata mặc định chưa cung cấp đầy đủ thông tin. Source analysis có thể được thực hiện đối với các OSS chưa có thông tin đầy đủ, trong khi binary analysis được sử dụng để bổ sung thông tin từ các binary trong root filesystem.

2.3.4 Dữ liệu đầu vào và đầu ra

FOSSLight Yocto Scanner sử dụng các dữ liệu được tạo ra từ WebOS/Yocto build làm input. Các input chính gồm:

Bảng 2.1: Dữ liệu đầu vào của FOSSLight Yocto Scanner.

Input	Vai trò
buildhistory/packages	Thông tin và lịch sử package trong quá trình build
bom.json	Thông tin BOM được tạo từ Yocto build
installed-package-names.txt	Danh sách package được cài đặt
installed-packages.txt	Thông tin chi tiết về package

Lệnh scan cơ bản có thể sử dụng:

```
fosslight_yocto \
  -p buildhistory/packages \
  -b bom.json \
  -i installed-package-names.txt \
  -ip installed-packages.txt \
  -o fosslight-result
```

Các tùy chọn như -a, -s, -c và -y được sử dụng khi cần thực hiện binary analysis, source analysis hoặc bổ sung thông tin OSS.

Sau khi scan, scanner tạo log và FOSSLight Report. Report thường được xuất dưới dạng Excel và là kết quả chính được sử dụng để phân tích OSS.

2.3.5 Giới hạn

FOSSLight Yocto Scanner chủ yếu dựa trên Yocto build metadata và thông tin package trong root filesystem. Do đó, kết quả scan không nên được xem là SBOM hoàn chỉnh của toàn bộ WebOS platform.

Các thành phần như Linux Kernel, Bootloader và Firmware nằm ngoài phạm vi package của root filesystem cần được kiểm tra và bổ sung thông tin riêng khi cần.

Ngoài ra, binary analysis và source analysis chỉ đóng vai trò bổ sung khi thông tin từ build metadata chưa đầy đủ. Đối với Yocto, build metadata từ recipe vẫn là nguồn thông tin quan trọng để xác định package, version và license.

2.4 So sánh công cụ

Ba công cụ được nghiên cứu trong OSSVS-2 có mục đích khác nhau trong quy trình quản lý SBOM và OSS. SBOM Manager tập trung vào quản lý và truy vấn SBOM, meta-wr-sbom tích hợp với Yocto để tạo SBOM, trong khi FOSSLight Yocto tập trung vào phân tích OSS từ metadata của Yocto build.

Bảng 2.2: So sánh các công cụ quản lý SBOM và OSS.

Công cụ	Vai trò	Input chính	Output	Điểm mạnh
SBOM Manager	Quản lý và truy vấn SBOM	SPDX, CycloneDX và các dữ liệu SBOM hỗ trợ	Repository và thông tin truy vấn	Quản lý tập trung nhiều SBOM
meta-wr-sbom	Tạo SBOM trong Yocto	Yocto build metadata	SBOM, chủ yếu SPDX	Tích hợp trực tiếp với Yocto
FOSSLight Yocto	Phân tích OSS trong Yocto	Yocto metadata, bom.json, buildhistory và package information	FOSSLight Report	Chuyên biệt cho việc xác định OSS, version và license

Các công cụ trên không phải là những giải pháp thay thế trực tiếp cho nhau mà đảm nhiệm các giai đoạn khác nhau. meta-wr-sbom có thể được sử dụng ở giai đoạn tạo SBOM trong quá trình Yocto build. FOSSLight Yocto sử dụng build metadata để phân tích và xây dựng OSS inventory. Sau khi có SBOM, SBOM Manager có thể được sử dụng để lưu trữ, quản lý và truy vấn thông tin component.

2.5 Thực nghiệm FOSSLight Yocto trên WebOS

2.5.1 Môi trường

Thực nghiệm được thực hiện trên máy build WebOS sử dụng hệ thống build dựa trên Yocto/OpenEmbedded. FOSSLight Yocto Scanner được cài đặt và thực thi trực tiếp trên máy build, không cần cài đặt scanner vào WebOS image.

Trước khi thực hiện scan, cần xác định các thông tin của môi trường build gồm phiên bản WebOS, phiên bản Yocto/BitBake, machine và target image. Các thông tin này giúp xác định chính xác môi trường được sử dụng trong thực nghiệm và hỗ trợ khả năng tái lập kết quả.

Các thông tin thực tế được ghi nhận:

Bảng 2.3: Môi trường thực nghiệm FOSSLight Yocto.

Thành phần	Giá trị
WebOS version	webOS OSE 2.20.1.g
Yocto/BitBake version	BitBake 2.0.0 (Yocto 4.0/Kirkstone)
Machine	raspberrypi4_64
Target image	webos-image
FOSSLight Yocto version	4.0.6

2.5.2 Chuẩn bị build metadata

FOSSLight Yocto Scanner sử dụng metadata được tạo trong quá trình Yocto build làm dữ liệu đầu vào. Trước khi scan, cần kiểm tra sự tồn tại của buildhistory, bom.json và các file chứa thông tin package đã được cài đặt.

Các dữ liệu chính gồm:

Bảng 2.4: Build metadata được sử dụng trong thực nghiệm.

File/Directory	Vai trò
buildhistory/packages	Thông tin package từ build history
bom.json	Thông tin BOM được tạo trong quá trình build
installed-package-names.txt	Danh sách package được cài đặt
installed-packages.txt	Thông tin package được cài đặt

Trong trường hợp chưa có buildhistory, Yocto có thể được cấu hình thông qua local.conf với buildhistory. Sau khi build image, write_bom_info được sử dụng để tạo bom.json.

2.5.3 Kiểm tra dữ liệu đầu vào

Các file đầu vào được kiểm tra bằng:

```
find . -name bom.json
find . \
  \( -name "installed-package-names.txt" \
  -o -name "installed-packages.txt" \)
```

2.5.4 Chạy scanner

Sau khi chuẩn bị đầy đủ dữ liệu đầu vào, tạo thư mục riêng để lưu kết quả:

```
mkdir -p fosslight-result
```

Thực hiện scan bằng lệnh:

```
fosslight_yocto \
  -p ~/build-webos/buildhistory/packages/raspberrypi4_64-webos-linux \
  -b ~/build-webos/bom.json \
  -i ~/build-webos/buildhistory/images/raspberrypi4_64/glibc/webos-image/
   installed-package-names.txt \
  -ip ~/build-webos/buildhistory/images/raspberrypi4_64/glibc/webos-image/
   installed-packages.txt \
  -o ~/build-webos/fosslight-result
```

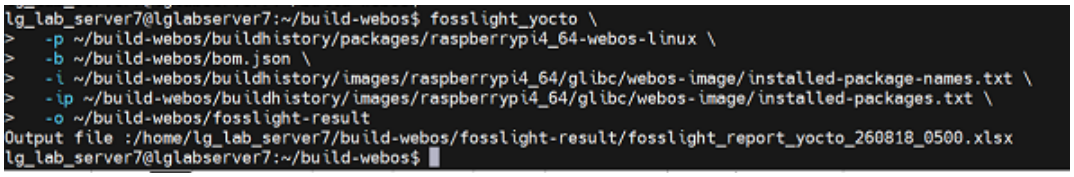
Trong đó, -p chỉ định build history của package, -b chỉ định file BOM, -i và -ip chỉ định thông tin package đã cài đặt, còn -o chỉ định thư mục chứa kết quả scan.

Sau khi scanner hoàn thành, kiểm tra kết quả:

```
find fosslight-result -type f
```

Kết quả cần ghi nhận trực tiếp từ môi trường thực nghiệm, không sử dụng tên file hoặc dữ liệu minh họa.

2.5.5 Kết quả

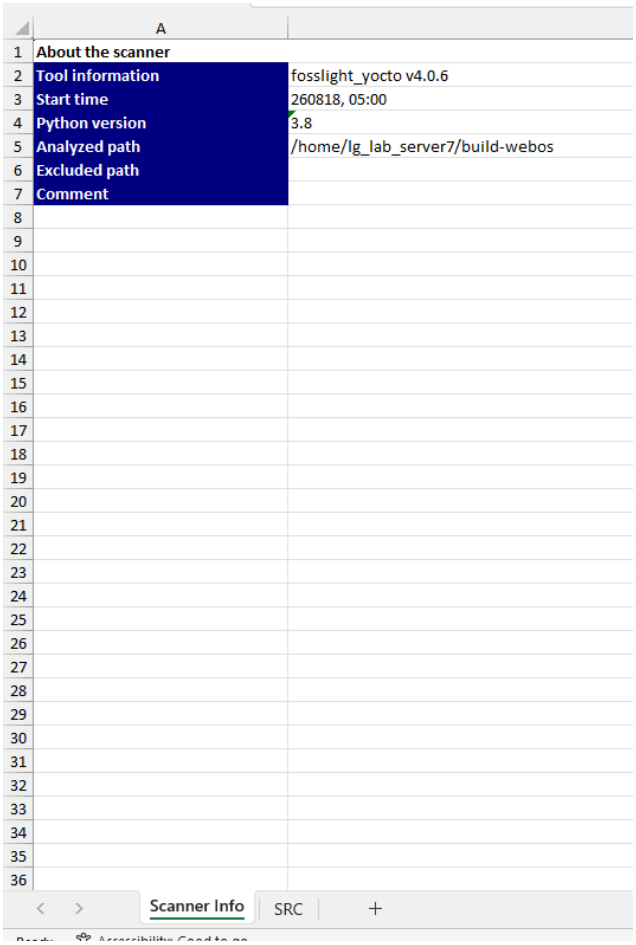


Hình 2.1: FOSSLight Yocto Scanner hoàn thành và tạo báo cáo.

FOSSLight Yocto Scanner phiên bản 4.0.6 được thực thi trên máy Ubuntu sử dụng để build WebOS. Scanner phân tích thư mục WebOS build tại /home/lg_lab_server7/build-webos và tạo FOSSLight Report dưới dạng Excel.

Kết quả được lưu tại:

```
/home/lg_lab_server7/build-webos/fosslight-result/
‘-- fosslight_report_yocto_260818_0500.xlsx
```



Hình 2.2: Sheet Scanner Info trong FOSSLight Report.

Report gồm hai sheet chính là Scanner Info và SRC. Sheet Scanner Info lưu thông tin về phiên bản scanner, Python và đường dẫn được phân tích. Sheet SRC chứa thông tin về các thành phần OSS được xác định trong quá trình scan.

Kết quả scan thu được 1.375 records trong sheet SRC, tương ứng với 444 OSS Name khác nhau. Mỗi record cung cấp các trường thông tin như OSS Name, OSS Version, License, Download Location và Source Path.

Như vậy, FOSSLight Yocto đã thực hiện thành công quá trình phân tích WebOS build và tạo được OSS inventory chứa thông tin về tên và phiên bản của các thành phần Open Source Software.

2.5.6 Phân tích OSS Version

Kết quả từ FOSSLight Report cho thấy scanner có thể xác định phiên bản của nhiều thành phần OSS được sử dụng trong WebOS. Thông tin phiên bản được thể hiện trong trường OSS Version của sheet SRC.

Trong tổng số 1.375 records được tạo ra, 1.367 records có thông tin OSS Version, trong khi 8 records chưa xác định được phiên bản. Như vậy, khoảng 99,4% records trong kết quả scan có thông tin phiên bản OSS.

Một số thành phần tiêu biểu được trích xuất từ kết quả scan:

Bảng 2.5: Một số thành phần OSS tiêu biểu trong kết quả scan.

OSS Name	OSS Version	License
openssl	3.0.7	apache-2.0
curl	7.82.0	curl
busybox	1.35.0	gpl-2.0-only, bzip2-1.0.4
zlib	1.2.11	zlib
glibc	2.35	gpl-2.0-only, lgpl-2.1-only
libxml2	2.9.14	mit
sqlite3	3.38.5	pd
systemd	250.5	gpl-2.0-only, lgpl-2.1-only
wayland	1.20.0	mit

Các kết quả trên được lấy trực tiếp từ FOSSLight Report được tạo trong quá trình thực nghiệm.

Một số OSS xuất hiện ở nhiều package hoặc source path khác nhau nhưng vẫn được quy về cùng một OSS Name và OSS Version. Ví dụ, openssl xuất hiện ở nhiều source path như libcrypto, libssl, openssl-bin và openssl-conf, nhưng đều được xác định là OpenSSL phiên bản 3.0.7.

Kết quả này cho thấy FOSSLight Yocto có khả năng tổng hợp thông tin từ các package trong Yocto build thành OSS inventory, đồng thời cung cấp phiên bản tương ứng của các thành phần OSS.

2.5.7 Đánh giá

Thực nghiệm cho thấy FOSSLight Yocto phù hợp với môi trường WebOS sử dụng Yocto vì scanner có thể khai thác trực tiếp build metadata để xây dựng OSS inventory. Kết quả cung cấp các thông tin quan trọng như OSS Name, OSS Version, License và Download Location, đáp ứng mục tiêu xác định thành phần OSS và phiên bản được sử dụng trong WebOS.

Tuy nhiên, kết quả FOSSLight Yocto không nên được xem là SBOM hoàn chỉnh của toàn bộ WebOS. Scanner chủ yếu xử lý package và metadata liên quan đến root filesystem. Các thành phần như Linux Kernel, Bootloader và Firmware ngoài rootfs cần được kiểm tra và bổ sung riêng khi cần thiết.

Do đó, FOSSLight Yocto phù hợp để sử dụng như một thành phần trong quy trình OSS inventory của WebOS. Kết quả scan có thể tiếp tục được sử dụng cho các hoạt động quản lý SBOM, license compliance và vulnerability analysis.

CHƯƠNG 3

Thu thập CVE và phân loại lỗ hổng

3.1 OSS được lựa chọn

Bảng 3.1: Các thành phần OSS được lựa chọn để phân tích.

OSS	Version	Nguồn
OpenSSL	3.0.7	FOSSLight Report
libxml2	2.9.14	FOSSLight Report
curl	7.82.0	FOSSLight Report



Hình 3.1: Thông tin OpenSSL 3.0.7 trong FOSSLight Report.



Hình 3.2: Thông tin libxml2 2.9.14 trong FOSSLight Report.



Hình 3.3: Thông tin curl 7.82.0 trong FOSSLight Report.

3.2 Thu thập CVE

3.2.1 OpenSSL 3.0.7

Bảng 3.2: CVE liên quan đến OpenSSL 3.0.7.

CVE	CVSS	Affected	Fixed version
CVE-2023-0286	7.4	Yes	3.0.8
CVE-2023-0464	7.5	Yes	3.0.9

3.2.2 libxml2 2.9.14

Bảng 3.3: CVE liên quan đến libxml2 2.9.14.

CVE	CVSS	Affected	Fixed/Patch
CVE-2022-40304	7.8	Yes	Security patch

3.2.3 curl 7.82.0

Bảng 3.4: CVE liên quan đến curl 7.82.0.

CVE	CVSS	Affected	Fixed version
CVE-2022-27774	5.7	Yes	—
CVE-2023-38545	9.8	Yes	8.4.0

3.3 Phân loại CVE

3.3.1 OpenSSL – CVE-2023-0286

Bảng 3.5: Kết quả phân loại CVE-2023-0286.

Nội dung	Kết quả
Version	3.0.7
CVSS	7.4
Affected	Yes
Fixed	3.0.8
Patch trên WebOS	Chưa xác định / Not confirmed
Mitigation	Upgrade $\geq$ 3.0.8 hoặc backport patch

Kiểm tra patch:

```
grep -Rni "CVE-2023-0286" meta-* 2>/dev/null
```



Hình 3.4: Kết quả tìm kiếm patch CVE-2023-0286 trong các layer WebOS.

Không tìm thấy patch được định danh trực tiếp bằng CVE-2023-0286 trong các layer được kiểm tra.

3.3.2 OpenSSL – CVE-2023-0464

Bảng 3.6: Kết quả phân loại CVE-2023-0464.

Nội dung	Kết quả
OSS	OpenSSL
Version	3.0.7
CVE	CVE-2023-0464
CVSS	7.5
Affected	Yes
Fixed version	3.0.9
WebOS patch status	Chưa xác định
Mitigation đề xuất	Upgrade ≥ 3.0.9 hoặc backport security patch

Kiểm tra patch trong WebOS:

```
grep -Rni "CVE-2023-0464" meta-* 2>/dev/null
```



Hình 3.5: Kết quả tìm kiếm patch CVE-2023-0464 trong các layer WebOS.

Không tìm thấy patch CVE-2023-0464 trong các layer meta-* được kiểm tra.

3.3.3 libxml2 – CVE-2022-40304

Bảng 3.7: Kết quả phân loại CVE-2022-40304.

Nội dung	Kết quả
OSS	libxml2
Version	2.9.14
CVE	CVE-2022-40304
CVSS	7.8
Affected	Yes
Fixed/Patch	Security patch
WebOS patch status	Chưa xác định
Mitigation đề xuất	Upgrade hoặc backport security patch

Kiểm tra patch trong WebOS:

```
grep -Rni "CVE-2022-40304" meta-* 2>/dev/null
```



Hình 3.6: Kết quả tìm kiếm patch CVE-2022-40304 trong các layer WebOS.

Không tìm thấy patch được định danh trực tiếp bằng CVE-2022-40304 trong các layer được kiểm tra.

3.3.4 curl – CVE-2023-38545

Bảng 3.8: Kết quả phân loại CVE-2023-38545.

Nội dung	Kết quả
OSS	curl
Version	7.82.0
CVE	CVE-2023-38545
CVSS	9.8
Affected	Yes
Fixed version	8.4.0
WebOS patch status	Chưa xác định
Mitigation đề xuất	Upgrade $\geq$ 8.4.0 hoặc backport security patch

```
grep -Rni "CVE-2023-38545" meta-* 2>/dev/null
```



Hình 3.7: Kết quả tìm kiếm patch CVE-2023-38545 trong các layer WebOS.

Không tìm thấy patch được định danh trực tiếp bằng CVE-2023-38545 trong các layer được kiểm tra.

3.4 Quản lý lỗ hổng

Đây là kết quả cuối cùng của OSSVS-3:

Bảng 3.9: Danh sách ưu tiên xử lý lỗ hổng.

OSS	Phiên bản	CVE	CVSS	Trạng thái	Xử lý	Ưu tiên
OpenSSL	3.0.7	CVE-2023-0286	7.4	Chưa xác định	Upgrade / backport	P1
OpenSSL	3.0.7	CVE-2023-0464	7.5	Chưa xác định	Upgrade / backport	P1
libxml2	2.9.14	CVE-2022-40304	7.8	Chưa xác định	Upgrade / backport	P1
curl	7.82.0	CVE-2023-38545	9.8	Chưa xác định	Upgrade / backport	P1 – Critical

3.5 Kết luận

OSSVS-3 đã xác định các CVE có CVSS ≥ 7.0 liên quan đến OpenSSL 3.0.7, libxml2 2.9.14 và curl 7.82.0 dựa trên OSS inventory từ OSSVS-2. Các CVE được thực hiện triage bằng cách đối chiếu version, affected range và kiểm tra patch được định danh trong các layer WebOS. Các trường hợp chưa xác định được trạng thái patch được đưa vào danh sách cần xử lý. Phương án upgrade hoặc backport được đề xuất làm mitigation và sẽ được thực hiện, kiểm chứng bằng rebuild trong OSSVS-4.

CHƯƠNG 4

Khắc phục lỗ hổng và xác minh cuối cùng

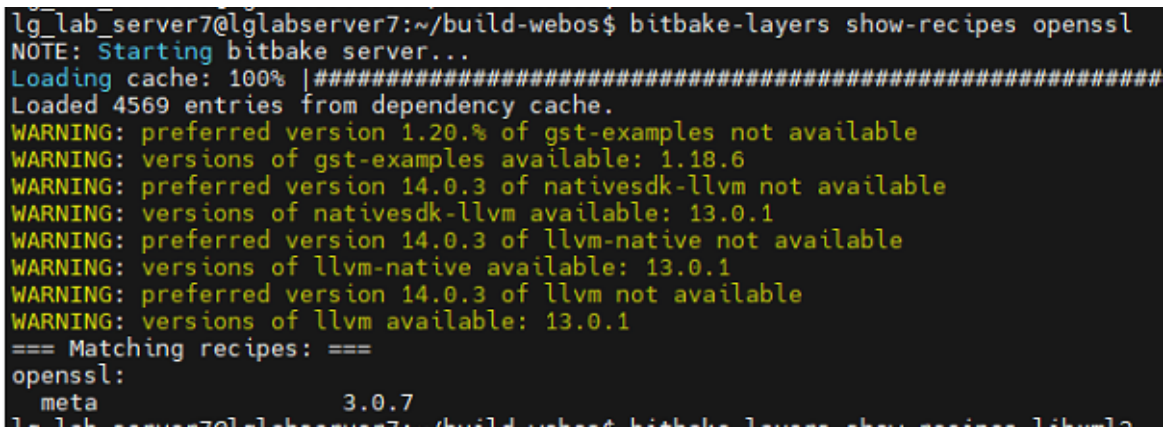
Chương này tiếp nối kết quả phân loại ở Chương 3. Ba thành phần OpenSSL, libxml2 và curl được xử lý theo phương án backport bản vá, sau đó được build riêng và build lại toàn bộ WebOS image để xác minh.

4.1 OpenSSL

4.1.1 Phiên bản hiện tại

Bảng 4.1: Kế hoạch khắc phục OpenSSL.

OSS	Before	CVE	Target
OpenSSL	3.0.7	CVE-2023-0464	Backport



Hình 4.1: Phiên bản OpenSSL trước khi khắc phục.

4.1.2 Nâng cấp hoặc vá lỗi

```
lg_lab_server7@lglabserver7:~/build-webos/oe-core$ bitbake-layers show-layers | grep meta-local
meta-local              /home/lg_lab_server7/meta-local          6
lg_lab_server7@lglabserver7:~/build-webos/oe-core$ bitbake -e openssl | grep '^SRC_URI='
SRC_URI="http://www.openssl.org/source/openssl-3.0.7.tar.gz           file://run-ptest           file://0001-buildinfo-stri
p-sysroot-and-debug-prefix-map-from-co.patch           file://afalg.patch           file://0001-Configure-do-not-tweak-mips
-cflags.patch           file://CVE-2022-3996.patch           file://CVE-2023-0464.patch     file://CVE-2023-0465.patc
h     file://CVE-2023-0466.patch "
lg_lab_server7@lglabserver7:~/build-webos/oe-core$
```

Hình 4.2: Cấu hình layer và patch OpenSSL trong WebOS.

4.1.3 Kết quả build

```
Initialising tasks: 100% |#################################################################################| Time: 0:00:00
Sstate summary: Wanted 53 Local 0 Mirrors 0 Missed 53 Current 146 (0% match, 73% complete)
Removing 5 stale sstate objects for arch cortexa72: 100% |#################################################| Time: 0:00:00
Removing 4 stale sstate objects for arch x86_64: 100% |###################################################| Time: 0:00:00
Removing 37 stale sstate objects for arch raspberrypi4_64: 100% |#########################################| Time: 0:00:00
NOTE: Executing Tasks
NOTE: Tasks Summary: Attempted 771 tasks of which 695 didn't need to be rerun and all succeeded.
NOTE: Writing buildhistory
NOTE: Writing buildhistory took: 3 seconds
NOTE: Build completion summary:
NOTE:   do_populate_sysroot: 0.0% sstate reuse(0 setscene, 4 scratch)
NOTE:   do_deploy_source_date_epoch: 0.0% sstate reuse(0 setscene, 2 scratch)
NOTE:   do_package_qa: 0.0% sstate reuse(0 setscene, 1 scratch)
NOTE:   do_package: 0.0% sstate reuse(0 setscene, 20 scratch)
NOTE:   do_packagedata: 0.0% sstate reuse(0 setscene, 20 scratch)
NOTE:   do_package_write_ipk: 0.0% sstate reuse(0 setscene, 5 scratch)
NOTE:   do_populate_lic: 0.0% sstate reuse(0 setscene, 1 scratch)
lg_lab_server7@lglabserver7:~/build-webos$ S
```

Hình 4.3: Kết quả build OpenSSL sau khi bổ sung patch.

```
289:NOTE: Applying patch 'CVE-2023-0464.patch' (../meta-local/recipes-connectivity/openssl/openssl/CVE-2023-0464.patch)
290:DEBUG: Python function patch_do_patch finished
291:DEBUG: Python function do_patch finished
```

Hình 4.4: Log xác nhận áp dụng patch CVE-2023-0464.

4.2 libxml2

4.2.1 Phiên bản hiện tại

Bảng 4.2: Kế hoạch khắc phục libxml2.

OSS	Before	CVE	Target
libxml2	2.9.14	CVE-2022-40304	Backport

4.2.2 Nâng cấp hoặc vá lỗi

```
lg_lab_server7@lglabserver7:~/build-webos$ bitbake -e libxml2 | grep '^FILE='
FILE="/home/lg_lab_server7/build-webos/oe-core/meta/recipes-core/libxml/libxml2_2.9.14.bb"
lg_lab_server7@lglabserver7:~/build-webos$ bitbake -e libxml2 | grep '^SRC_URI='
SRC_URI="https://download.gnome.org/sources//libxml2/2.9/libxml2-2.9.14.tar.xz;name=archive http://www.w3.org/XML/Test/xmlts20080827.tar;subdir=libxml2-2.9.14;name=testtar file://libxml-64bit.patch file://runtest.patch
  file://run-ptest file://python-sitepackages-dir.patch file://0001-Make-ptest-run-the-python-tests-if-python-is-enabled.patch file://fix-execution-of-ptests.patch file://remove-fuzz-from-ptests.patch
      file://libxml-m4-use-pkgconfig.patch file://0001-Port-gentest.py-to-Python-3.patch file://CVE-2022-40303.patch file://CVE-2022-40304.patch "
```

Hình 4.5: Cấu hình recipe và patch cho libxml2.

4.2.3 Kết quả build

```
bitbake -c patch libxml2
```

```
Initialising tasks: 100% |#################################################################################| Time: 0:00:00
Sstate summary: Wanted 0 Local 0 Mirrors 0 Missed 0 Current 24 (0% match, 100% complete)
NOTE: Executing Tasks
NOTE: Tasks Summary: Attempted 111 tasks of which 108 didn't need to be rerun and all succeeded.
NOTE: Writing buildhistory
NOTE: Writing buildhistory took: 2 seconds
lg_lab_server7@lglabserver7:~/build-webos$
```

Hình 4.6: Kết quả build libxml2 sau khi bổ sung patch.

```
.py-to-Python-3.patch)
424-NOTE: Applying patch 'CVE-2022-40303.patch' (oe-core/meta/recipes-core/libxml/libxml2/CVE-2022-40303.patch)
425:NOTE: Applying patch 'CVE-2022-40304.patch' (oe-core/meta/recipes-core/libxml/libxml2/CVE-2022-40304.patch)
426-DEBUG: Python function patch.do_patch finished
```

Hình 4.7: Log xác nhận áp dụng patch CVE-2022-40304.

4.3 curl

4.3.1 Phiên bản hiện tại

Bảng 4.3: Kế hoạch khắc phục curl.

OSS	Before	CVE	Target
curl	7.82.0	CVE-2023-38545	Backport

4.3.2 Nâng cấp hoặc vá lỗi

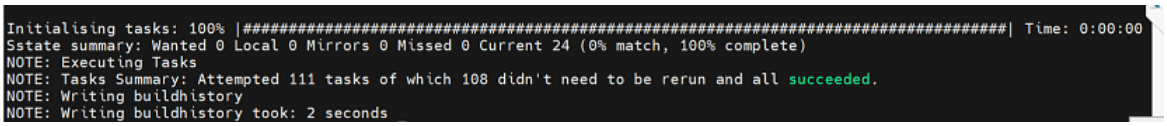
```
lg_lab_server7@lglabserver7:~/build-webos/oe-core$ cat ~/meta-local/recipes-support/curl/curl_7.82.0.bbappend
FILESEXTRAPATHS:prepend := "${THISDIR}/curl:"

SRC_URI += " \
    file://CVE-2023-38545.patch \
"
```

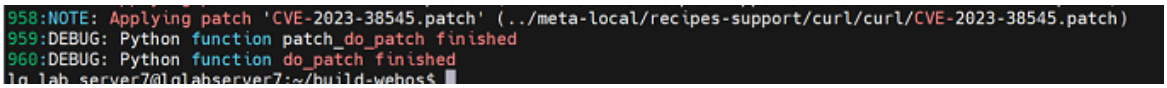
Hình 4.8: Cấu hình bbappend và patch cho curl.

4.3.3 Kết quả build

```
bitbake -c patch curl
```



Hình 4.9: Kết quả build curl sau khi bổ sung patch.



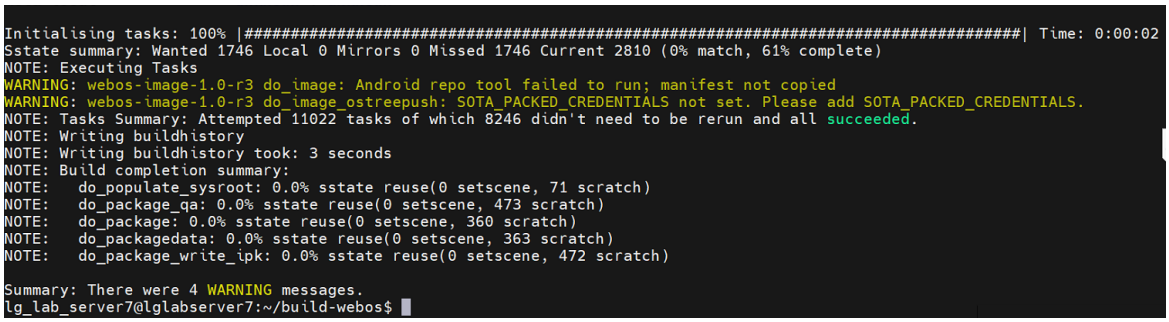
Hình 4.10: Log xác nhận áp dụng patch CVE-2023-38545.

4.4 Build lại WebOS

4.4.1 Lệnh build

```
bitbake webos-image
```

4.4.2 Kết quả build



Hình 4.11: Kết quả build lại WebOS image.

4.5 Xác minh

4.5.1 Phiên bản OSS sau khắc phục

Bảng 4.4: Phiên bản OSS trước và sau khi backport bản vá.

OSS	Before	After	CVE	Remediation
OpenSSL	3.0.7	3.0.7	CVE-2023-0464	Backport
libxml2	2.9.14	2.9.14	CVE-2022-40304	Backport
curl	7.82.0	7.82.0	CVE-2023-38545	Backport